Inhame: sistema de busca de receitas por ingredientes*

Inhame: an ingredient-based recipe search system

Ana Beatriz Costa Viana[1]
Gustavo Anselmo Santos Silva[2]
Josué Santana Melo[3]
Luan Magno Souza Alves[4]
Nicole Marie Agnelo Marques[5]

Resumo

Este artigo apresenta a proposta de desenvolvimento de um sistema de busca de receitas que tem como objetivo facilitar a escolha de refeições no dia a dia. A ferramenta permitirá que os usuários encontrem receitas com base nos ingredientes disponíveis em casa, além de filtros como tempo de preparo, tipo de refeição e nível de dificuldade. A ideia surgiu a partir da dificuldade que muitas pessoas enfrentam ao decidir o que cozinhar, buscando oferecer uma solução prática que ajude no aproveitamento dos alimentos e na descoberta de novas receitas. Na etapa de aquisição, foram reunidos e unificados 83.933 documentos com URLs distintas, provenientes de dez sites de receitas em português. Na etapa de representação, os documentos foram limpos, divididos em campos e deduplicados, resultando em 74.209 documentos, dos quais 66.872 foram identificados como receitas. Sobre eles foi implementado, sem banco de dados, um índice invertido posicional por campos, construído pelo método SPIMI e comprimido com Variable Byte, que ocupa 39,9 MiB e é construído em cerca de 90 segundos. Experimentos com 32 configurações avaliam o efeito da remoção de stopwords, do stemming, da granularidade e do tamanho dos lotes de indexação sobre o tempo, a memória e o espaço.

Palavras-chave: Sistema de busca; Receitas; Ingredientes; Tecnologia; Praticidade.

*Recuperação de Informação na Web, Sistemas de Informação, Unidade São Gabriel

[1]Graduanda em Sistemas de Informação pela PUC Minas, Brasil, Brasil– 1446728@pucminas.edu.br
[2]Graduando em Sistemas de Informação pela PUC Minas, Brasil, Brasil– gustavosilvasocial@gmail.com
[3]Graduando em Sistemas de Informação pela PUC Minas, Brasil, Brasil– josue.melo@sga.pucminas.br
[4]Graduando em Sistemas de Informação pela PUC Minas, Brasil, Brasil– lmsalves@pucminas.br
[5]Graduanda em Sistemas de Informação pela PUC Minas, Brasil, Brasil– 1440058@pucminas.br

Abstract

This article presents the proposal for developing a recipe search system designed to make everyday meal choices easier. The tool will allow users to find recipes based on the ingredients they have at home, as well as filters such as preparation time, meal type, and difficulty level. The idea emerged from the difficulty many people face when deciding what to cook, aiming to provide a practical solution that encourages better use of available ingredients and the discovery of new recipes. The acquisition stage produced a unified corpus of 83,933 documents with distinct URLs from ten Portuguese-language recipe websites. In the representation stage, the documents were cleaned, split into fields and deduplicated, yielding 74,209 documents, 66,872 of which were identified as recipes. A positional field-based inverted index was built over them without any database, using the SPIMI method and Variable Byte compression; it takes 39.9 MiB and is built in about 90 seconds. Experiments with 32 configurations assess how stopword removal, stemming, granularity and indexing block size affect time, memory and space.

Keywords: Search system; Recipes; Ingredients; Technology; Practicality.

1 INTRODUÇÃO

Atualmente, com a rotina cada vez mais corrida e a busca por praticidade, muitas pessoas acabam optando por pedir delivery por não terem tempo ou criatividade para pensar no que preparar para comer. Além disso, é comum ter ingredientes em casa e não saber exatamente quais receitas podem ser feitas com eles.

Pensando nisso, surgiu a ideia de desenvolver um sistema de busca de receitas que facilite esse processo. A ferramenta permitirá que o usuário encontre receitas com base nos ingredientes disponíveis, sem precisar saber o nome do prato ou passar muito tempo pesquisando.

2 JUSTIFICATIVA

A proposta surgiu a partir de uma dificuldade comum no dia a dia: decidir o que cozinhar de forma rápida e prática. O sistema busca facilitar essa escolha, ajudando os usuários a aproveitarem melhor os ingredientes que já possuem em casa e descobrirem novas opções de refeições.

Além disso, o projeto permite aplicar conhecimentos de desenvolvimento de sistemas na criação de uma solução para um problema cotidiano.

3 OBJETIVOS

3.1 Objetivo Geral

Desenvolver um sistema de busca de receitas que facilite a escolha de refeições com base nos ingredientes disponíveis e nas preferências do usuário.

- Permitir a busca de receitas por ingredientes.
- Disponibilizar filtros como tempo de preparo, tipo de refeição e nível de dificuldade.
- Apresentar os ingredientes e o modo de preparo das receitas.
- Facilitar a descoberta de novas opções de refeições.
- Incentivar o melhor aproveitamento dos alimentos disponíveis em casa.

3.2 Objetivo Específico

Desenvolver um sistema de recuperação de informação voltado à busca de receitas culinárias por ingredientes.

4 METODOLOGIA

4.1 Coletor de Dados

Para construir a base do sistema Inhame, foi implementado em Python um coletor vertical, direcionado a fontes de receitas. A proposta do sistema é recuperar receitas pelos ingredientes disponíveis e, futuramente, oferecer filtros de tempo de preparo, tipo de refeição e dificuldade. Nesta entrega, o produto obtido é a coleção de documentos para uma etapa posterior de indexação e busca; esses filtros ainda não foram avaliados. As dez fontes que contribuíram com documentos para a base centralizada estão apresentadas na Tabela 1.

Na primeira versão, utilizamos `aiohttp` nas requisições e `BeautifulSoup` na extração. Cada fonte possui sua própria fila. URLs candidatas são obtidas de sementes, de arquivos de mapa do site (*sitemaps*), quando disponíveis, e de links internos extraídos das páginas HTML. São priorizadas URLs que parecem conter receitas, e o rastreamento é limitado aos hosts e caminhos configurados.

Na coleta complementar, foi utilizada uma implementação com Scrapy e integração ao Camoufox para tentar renderizar páginas não reconhecidas a partir do HTML recebido. A configuração por fonte passou a incluir padrões para distinguir URLs de receitas individuais de listagens e caminhos institucionais. Essa ampliação acrescentou novas fontes, incluindo publicações em português de Portugal.

As páginas aceitas são armazenadas em arquivos JSONL separados por fonte, contendo URL, título, descrição, texto extraído, metadados, data de coleta e resumo criptográfico. Na primeira implementação, o resumo criptográfico corresponde ao HTML recebido, mas o HTML integral não é conservado na saída. A versão posterior também registra ingredientes, instruções e dados estruturados quando identificados. Os formatos foram preservados na unificação, sem preencher artificialmente os campos ausentes.

4.2 Propriedades do Coletor

A estratégia prioriza cobertura de receitas e execução concorrente entre fontes. Na primeira implementação, cada domínio possui quatro trabalhadores. Eles podem realizar requisições simultâneas, respeitando um intervalo mínimo entre os inícios das consultas ao mesmo

domínio. O coletor controla URLs repetidas dentro de cada fonte antes da requisição; o resumo criptográfico do conteúdo é registrado, mas não é utilizado nessa implementação para eliminar documentos iguais em URLs distintas.

Na implementação com Scrapy, a concorrência também é controlada globalmente e o ritmo de acesso é ajustado conforme as respostas das fontes. A quantidade de requisições é acompanhada separadamente da quantidade de documentos aceitos: respostas de erro, redirecionamentos e páginas rejeitadas não aumentam o corpus. Por se tratar de receitas, optou-se por ampliar a cobertura, sem estabelecer uma agenda de atualização contínua das páginas.

4.3 Política de Seleção

Na primeira implementação, a seleção ocorre após receber uma resposta HTTP 200 com conteúdo HTML. O coletor aceita uma página quando o idioma declarado está ausente ou começa por `pt_BR` e pelo menos uma destas condições é satisfeita: (i) dados estruturados JSON-LD contêm o tipo `Recipe`; (ii) a metatag `article:section` contém "receit"; ou (iii) a URL contém "receit" e a descrição contém um termo culinário, como "receita", "culinária", "alimentação", "menu" ou "prato". A descrição é lida nas metatags `og:description` ou `description`. O idioma é obtido de `og:locale` ou `locale`. A URL isolada não basta para a aceitação.

Na implementação posterior, o tratamento de idioma também contempla português de Portugal. A seleção combina padrões de caminhos com a identificação de nome da receita, ingredientes e instruções nos dados estruturados ou em seções do texto. Páginas de categorias e listagens podem ser percorridas para descobrir links, sem serem consideradas automaticamente documentos de receita.

A detecção automática é heurística; não foi feita uma avaliação manual da precisão de todo o corpus. Como as implementações adotam critérios diferentes, a unificação não equivale a uma validação integral das receitas. O total apresentado corresponde a documentos com URLs distintas, e não a receitas semanticamente únicas.

Tabela 1 – Fontes com documentos presentes no corpus unificado

Fonte	Domínio
Ana Maria Brogui	`anamariabrogui.com.br`
Estadão Paladar	`estadao.com.br/paladar`
Panelinha	`panelinha.com.br`
Petitchef	`pt.petitchef.com`
Receitas Globo	`receitas.globo.com`
Receitas de Minuto	`receitasdeminuto.com`
TeleCulinária	`teleculinaria.pt`
Terra Degusta	`terra.com.br/vida-e-estilo/degusta/receitas/`
TudoGostoso	`tudogostoso.com.br`
TudoReceitas	`tudoreceitas.com`

4.4 Política de Revisitação

Durante a execução, URLs normalizadas que já foram processadas em uma fila não são novamente requisitadas pela mesma fonte. Não foi implementada uma agenda de revisitação para ciclos futuros. O histórico de URLs também não é compartilhado automaticamente entre execuções independentes, de modo que uma nova execução pode visitar páginas já coletadas anteriormente.

Por esse motivo, os arquivos das coletas selecionadas passaram por uma deduplicação adicional durante a preparação da base centralizada. O procedimento e suas regras são apresentados na Seção 5.1.2.

4.5 Tolerâncias

Na execução inicial, configuraram-se quatro trabalhadores por domínio, intervalo mínimo de 0,25 segundo entre inícios de requisições do mesmo domínio e tempo limite de 15 segundos por requisição. O coletor consulta `robots.txt`, verifica as permissões de acesso e adota eventual `Crawl-delay` maior que o intervalo configurado. URLs impedidas por essas regras são contabilizadas separadamente. Nessa implementação, não há repetição imediata das requisições que falham nem redução automática da taxa após respostas HTTP 429. Essas respostas são contabilizadas como falhas HTTP, e não como páginas rejeitadas pelo classificador.

Na implementação posterior, foram configuradas até 72 requisições HTTP simultâneas no total e até quatro por fonte, com intervalo mínimo de 0,5 segundo e partida em um segundo. O AutoThrottle ajusta o intervalo conforme a latência das respostas. Respostas 403, 429 ou 503 interrompem novas consultas à fonte afetada naquela execução. Esses limites procuram distribuir o trabalho entre fontes, mas não garantem ausência de bloqueios.

A integração ao Camoufox foi prevista como alternativa para páginas dependentes de renderização. Entretanto, sua presença no código não comprova que todas essas páginas tenham sido processadas com sucesso. A cobertura efetiva é avaliada pelos documentos presentes nos arquivos de saída.

4.6 Critério de Parada

Na execução inicial, definimos o limite de 52.000 documentos aceitos ou uma janela de coleta de 90 minutos, encerrando também caso as filas de URLs fossem esgotadas. Nas coletas posteriores, a entrada interativa permite definir uma meta de documentos e um limite opcional de tempo. O encerramento também pode ser solicitado pelo usuário.

Assim, uma execução sem limite de tempo continua enquanto houver URLs elegíveis para processar, sem repetir indefinidamente o rastreamento. A preparação da base considerou o

conteúdo efetivamente recebido nas duas saídas selecionadas. O corpus resultante possui 83.933 documentos com URLs distintas, superando a referência quantitativa de 50 mil documentos adotada para a aquisição.

5 RESULTADOS E ETAPAS DE DESENVOLVIMENTO

5.1 Resultados da Coleta e Preparação do Corpus

5.1.1 Consolidação das Saídas

Na execução inicial em um computador pessoal, foram aceitas 42.288 páginas a partir de 73.942 URLs requisitadas, ao longo de 90,02 minutos. A taxa média global foi de 13,69 requisições e 7,83 documentos aceitos por segundo. A razão entre documentos aceitos e requisições foi de 57,2%; ela não constitui uma medida de precisão do classificador, pois inclui requisições com erros HTTP. Esses números descrevem aquela execução e não o corpus acumulado apresentado neste relatório.

Para centralizar os dados, utilizamos os arquivos JSONL presentes em duas saídas: a coleta da primeira implementação e a coleta complementar com novas fontes. A primeira continha 45.107 registros, enquanto a segunda continha 41.498, totalizando 86.605 registros antes da deduplicação.

As contagens foram calculadas diretamente nos arquivos de conteúdo. Essa escolha é importante porque o relatório de uma execução da primeira implementação registra 42.288 documentos em aproximadamente 90 minutos, mas os arquivos entregues daquela coleta contêm 45.107 registros. Portanto, o total do diretório não foi atribuído ao tempo de uma única execução. Na coleta complementar, o relatório disponível ainda estava marcado como parcial, embora sua contagem de 41.498 documentos coincidisse com os arquivos recebidos. Por esse motivo, não foi calculada uma taxa global de coleta a partir da soma dos relatórios.

5.1.2 Remoção de Registros com URL Repetida

A deduplicação utilizou o campo `url` como chave, retirando somente espaços nas extremidades. Quando a mesma URL aparecia mais de uma vez, foi preservado o registro com a data de coleta mais recente. Em caso de empate, foram considerados o preenchimento dos campos, o tamanho do texto e, por último, a primeira ocorrência nos arquivos de entrada.

Foram removidos 2.672 registros com URL repetida, resultando em 83.933 documentos. URLs com diferenças de protocolo, parâmetros, domínio ou barra final não foram automaticamente consideradas equivalentes. Também não foi aplicada, nesta preparação, uma remoção

por semelhança textual. Assim, páginas com URLs diferentes que apresentem a mesma receita ainda podem estar presentes no corpus.

Cada documento recebeu um identificador sequencial no campo `doc_id`. O campo `provenance` registra o arquivo de origem e a linha correspondente. Os campos de conteúdo foram preservados, permitindo rastrear cada registro até a saída original. A Tabela 2 apresenta o resultado por fonte.

Tabela 2 – Registros por fonte antes e depois da deduplicação por URL

Fonte	Lidos	Removidos	Mantidos
Ana Maria Brogui	9	0	9
Estadão Paladar	2.675	0	2.675
Panelinha	11.063	853	10.210
Petitchef	29.074	0	29.074
Receitas Globo	16.398	492	15.906
Receitas de Minuto	1.502	0	1.502
TeleCulinária	10.913	0	10.913
Terra Degusta	63	42	21
TudoGostoso	2.833	272	2.561
TudoReceitas	12.075	1.013	11.062
Total	**86.605**	**2.672**	**83.933**

5.1.3 Organização e Conferência da Base

O conjunto centralizado foi salvo em `corpus.jsonl`, em UTF-8, com um documento por linha. Esse formato permite percorrer os registros sequencialmente na etapa de indexação, sem exigir o carregamento de todo o arquivo na memória nem o uso de um sistema de banco de dados.

A conferência do arquivo final confirmou 83.933 objetos JSON válidos, 83.933 URLs distintas, identificadores sequenciais e texto preenchido em todos os registros. O arquivo ocupa 550.910.265 bytes, aproximadamente 525,39 MiB. Essa verificação confirma a integridade estrutural do corpus, mas não substitui a análise do conteúdo de cada página.

Para documentar a preparação, também foram gerados um manifesto com contagens e hash SHA-256, um inventário por fonte e uma tabela de auditoria com os registros removidos e suas versões mantidas. O código utilizado na unificação foi preservado junto aos arquivos, permitindo reproduzir o procedimento a partir das duas saídas originais.

A estrutura dos documentos varia conforme a implementação de origem. Os registros mantêm campos como URL, título, descrição, texto, fonte e data de coleta. Parte deles também possui ingredientes, instruções e dados estruturados da receita. Os campos que não estavam disponíveis na coleta original não foram preenchidos artificialmente durante a unificação.

5.1.4 Encaminhamento para a Representação e Indexação

Com o corpus centralizado, a etapa seguinte aplicou a limpeza dos textos, avaliou a presença de conteúdo fora do domínio e tratou as repetições de conteúdo que não foram identificadas pela comparação de URLs. Em seguida, foram realizadas a análise léxica, a remoção de stopwords e o stemming para a construção do índice invertido.

Os experimentos compararam transformações léxicas, granularidade e tamanho dos lotes de processamento, medindo tempo, memória e tamanho do índice. Essas etapas são descritas na Seção 5.2.

5.2 Representação e Indexação

5.2.1 Visão Geral da Solução

A entrada desta etapa é o arquivo `corpus.jsonl`, com os 83.933 documentos da unificação. Cada página é representada por cinco campos: **título**, **descrição**, **ingredientes**, **modo de preparo** e **corpo**, este último formado pelo texto restante depois da limpeza. O processamento, implementado em Python e entregue na pasta `parte2/`, tem quatro etapas:

1. **limpeza**: separa os campos, remove menus e rodapés, identifica as receitas e elimina cópias;
2. **análise léxica**: divide o texto em termos, remove stopwords e aplica stemming;
3. **indexação**: constrói o índice invertido e mede tempo, memória e espaço;
4. **experimentos**: compara 32 configurações e gera as figuras e tabelas deste relatório.

O índice é gravado em arquivos próprios, sem banco de dados ACID nem NoSQL. Essa escolha se justifica pelo volume e pela forma de acesso: o índice principal ocupa 39,9 MiB e seu vocabulário cabe inteiro em memória. A construção lê os documentos uma única vez, em sequência, e a busca lê cada lista de ocorrências com um único acesso ao disco. Essas operações não exigem transações nem escrita concorrente, que justificariam um banco de dados.

A Tabela 3 resume as decisões tomadas nesta etapa, detalhadas nas seções seguintes.

Tabela 3 – Resumo das decisões da representação

Aspecto	Decisão	Motivo
Campos	título, descrição, ingredientes, preparo e corpo	permite buscar só nos ingredientes
Limpeza	remoção de menus, rodapés e cópias	o texto caiu 50% e 9.724 cópias foram removidas
Páginas que não são receitas	mantidas, com marcação	a exclusão será decidida pela avaliação da busca
Stopwords	lista do NLTK e 36 termos do domínio	menor índice entre as listas comparadas
Stemming	RSLP (provisório)	menor vocabulário; será comparado ao redutor de plurais
Unidade indexada	documento, com termos separados por campo	receitas são curtas; trechos aumentam o índice
Posições	armazenadas	necessárias para buscar expressões como "leite condensado"
Compressão	Variable Byte	listas 3,5 vezes menores
Construção	SPIMI, lotes de 500 mil postings	menor uso de memória, mesmo índice

5.2.2 Limpeza e Padronização dos Documentos

As duas implementações do coletor produziram documentos com estruturas diferentes. Na primeira, cada página tem apenas o texto corrido, que inclui menus, rodapés, anúncios e listas de links para outras páginas: no Receitas Globo, a mediana do texto é de 7.555 caracteres. Na segunda, ingredientes e passos estão em campos próprios, e o texto do Petitchef tem mediana de 251 caracteres porque contém só os ingredientes. Indexar o texto bruto faria o tamanho de um documento depender mais do site que da receita; por isso, a limpeza converte todas as páginas para os mesmos cinco campos.

A limpeza trata marcação HTML residual, caracteres especiais, espaços repetidos e frases de publicidade ("Continua após a publicidade"). Na coleta complementar, ingredientes e passos são lidos dos campos estruturados. Na primeira coleta, eles são localizados por marcadores definidos para cada site: onde começam os ingredientes ("Ingredientes"), onde começa o preparo ("Modo de preparo", "Preparo" ou "Como fazer . . . :") e onde termina a receita ("Compartilhar", "Veja também", "Valor nutricional" etc.). Os trechos levados para ingredientes e preparo são retirados do corpo, para não serem contados duas vezes, e o nome do site é retirado do título ("Receita de Pizza, enviada por . . . – TudoGostoso" passa a "Pizza").

Menus e rodapés (*boilerplate*) são removidos por um método estatístico, que não depende de marcadores: trechos de oito palavras que se repetem em pelo menos 20% das páginas de um mesmo site são considerados parte do layout e removidos de todas as páginas desse site.

No TudoGostoso, 70% do texto era boilerplate; no Petitchef, cujo texto não tem navegação, nada foi removido. Além disso, o corpo é cortado no início das listas de links, como "Receitas relacionadas" e "Últimas matérias".

A identificação de documentos fora do domínio é feita por uma regra simples: a página é marcada como receita quando tem uma lista de ingredientes e um modo de preparo com tamanho mínimo; na coleta complementar, exige-se também a classificação de receita feita pelo coletor. A marcação de receita nos dados estruturados (JSON-LD) não pôde ser usada sozinha, pois o coletor não a encontrou em nenhuma das 15.906 páginas do Receitas Globo, nem mesmo nas receitas. A regra foi conferida por inspeção de amostras: listagens ("Receitas de Berinjela – 95 receitas"), matérias e posts de blog foram marcados como não receitas, e as receitas tiveram ingredientes e preparo separados corretamente. Não houve uma avaliação manual sistemática, e alguns posts de blog com uma receita completa são marcados como receita. As páginas que não são receitas foram mantidas com a marcação, para que a decisão de excluí-las seja tomada com a avaliação da busca.

As repetições de conteúdo em URLs diferentes são removidas: dois documentos são considerados cópias quando têm o mesmo endereço depois de padronizado (sem "www", sem diferença entre HTTP e HTTPS e sem barra final) ou o mesmo título, ingredientes e preparo. De cada grupo de cópias é mantido o documento mais completo.

Os campos ausentes na primeira coleta não foram inventados: ingredientes e preparo são extraídos do texto pelos marcadores e, quando não são encontrados, ficam vazios. Também foram extraídos metadados para os filtros previstos (tempo de preparo, rendimento, dificuldade e categoria), lidos dos dados estruturados ou do texto de cada site (como "Nível Fácil Tempo 40min"). O tempo de preparo foi obtido para 54.554 documentos e a dificuldade para 16.541. A Tabela 4 apresenta as contagens em cada etapa; o resultado por fonte está na Tabela 11, no Apêndice A.

Tabela 4 – Quantidade de documentos ao longo da preparação para indexação

Etapa	Documentos restantes
Entrada unificada, sem URLs repetidas	83.933
Após tratamento de documentos vazios ou inválidos (nenhum encontrado)	83.933
Após tratamento de duplicatas de conteúdo e de URLs equivalentes (9.724 removidos)	74.209
Após avaliação do conteúdo de receitas (66.872 marcados como receita; os demais mantidos com marcação)	74.209
Corpus final utilizado na indexação	74.209

Das 9.724 cópias removidas, 7.273 estavam no Panelinha, que publicava a mesma receita em variações de endereço (HTTP e HTTPS, com e sem "www" e em maiúsculas). Das 66.872 receitas, 25.385 vêm da primeira coleta e 41.487 da coleta complementar. O texto total caiu de 312,8 para 156,0 milhões de caracteres, uma redução de 50,1%.

5.2.3 Análise Léxica e Normalização dos Termos

A tokenização utiliza uma expressão regular própria, que separa o texto em palavras (incluindo as acentuadas) e números (inteiros, decimais e frações). Antes dela, o texto é convertido para minúsculas, e frações como "½" são convertidas para "1/2". Pontuação e hifens separam os tokens: "pimenta-do-reino" gera três tokens, e a busca pela expressão continua possível graças às posições. Os acentos são removidos depois do stemming, pois usuários costumam digitar "acucar" em vez de "açúcar". O mesmo processamento é aplicado aos documentos e às consultas, para que os termos coincidam.

Os números e frações das quantidades, como "1/2" e "200", são descartados, porque raramente são buscados e aumentariam o vocabulário em 21,8%. Abreviações de uma letra, como o "g" de "200 g", também são descartadas. Unidades por extenso, como "colher" e "xícara", são mantidas. A posição de cada termo conta também as palavras descartadas; assim, em "pão de queijo", "queijo" fica duas posições depois de "pão", tanto no documento quanto na consulta.

Um exemplo do processo é apresentado na Tabela 5, a partir do início do campo de ingredientes do documento 2.781 do corpus limpo, "Caldinho de feijão-preto com pimentão", do Panelinha. Após a tokenização e a normalização, os 19 termos resultantes são: `xic cha feij pret coz cald xic cha cald legum cas agu piment vermelh cebol dent alh colh sop`.

5.2.4 Remoção de Stopwords e Stemming

A lista de stopwords é a do NLTK para o português, com 207 palavras, acrescida de 36 termos frequentes do domínio que pouco distinguem uma receita de outra, como "receita", "ingredientes", "modo", "preparo", "gosto", "misture" e "reserve". Foram comparadas três opções: nenhuma remoção, a lista do NLTK e a lista do NLTK com os termos do domínio.

Unidades por extenso ("colher", "xícara", "pitada") foram preservadas, pois não estão na lista e podem distinguir preparos. As negações, porém, são removidas, pois "sem" e "não" fazem parte da lista do NLTK. Com isso, a consulta "bolo sem ovo" é reduzida aos termos `bol ovo`, e a restrição se perde. Essa limitação foi identificada depois dos experimentos e será corrigida na próxima etapa, mantendo as negações fora da lista. Da mesma forma, a lista do domínio inclui "leve" como verbo ("leve ao forno"), embora a palavra também possa ser adjetivo ("receita leve").

O stemming, que reduz as palavras a um radical comum, é feito com o RSLP (ORENGO; HUYCK, 2001), um algoritmo projetado para o português, disponível no NLTK. Também foram comparados o Snowball para o português (PORTER, 2001) e um redutor de plurais implementado pelo grupo, que trata apenas plurais regulares ("cebolas" $\rightarrow$ "cebola", "pães" $\rightarrow$ "pão").

O stemming é aplicado depois da remoção de stopwords e antes da remoção de acentos. A Tabela 5 mostra a saída real de cada etapa para o trecho do documento 2.781.

Tabela 5 – Exemplo das transformações aplicadas a um trecho do corpus (documento 2.781)

Etapa	Resultado observado
Texto de entrada	2 xícaras (chá) de feijão-preto cozido (com o caldo) 1½ xícara (chá) de caldo de legumes caseiro (ou água) ½ pimentão vermelho ½ cebola 1 dente de alho 1 colher (sopa) de
Tokenização e normalização	2 xícaras chá de feijão preto cozido com o caldo 1 1/2 xícara chá de caldo de legumes caseiro ou água 1/2 pimentão vermelho 1/2 cebola 1 dente de alho 1 colher sopa de
Descarte de números e tokens de uma letra	xícaras chá de feijão preto cozido com caldo xícara chá de caldo de legumes caseiro ou água pimentão vermelho cebola dente de alho colher sopa de
Remoção de stopwords	xícaras chá feijão preto cozido caldo xícara chá caldo legumes caseiro água pimentão vermelho cebola dente alho colher sopa
Stemming (RSLP)	xíc chá feij pret coz cald xíc chá cald legum cas águ piment vermelh cebol dent alh colh sop
Remoção de acentos (termos do índice)	xic cha feij pret coz cald xic cha cald legum cas agu piment vermelh cebol dent alh colh sop

O exemplo mostra o efeito de cada etapa: os 34 tokens são reduzidos a 19 termos, e "xícaras" e "xícara" passam a ser o mesmo termo. Também mostra uma limitação do RSLP: "caseiro" é reduzido a "cas", o mesmo radical de "casa". A Tabela 6 mede esse comportamento sobre as 31.913 palavras que aparecem pelo menos cinco vezes na coleção. O RSLP é o mais agressivo e reduz o vocabulário em 52,1%, reunindo, por exemplo, "assado", "assar", "asse" e "assando" em "ass". Essa agressividade também produz agrupamentos indevidos: "leite" é reunido com "leitura" e "leitor", "massa" com "massagem" e "cebola" com "cebolinha", que é outro ingrediente. O Snowball tem comportamento parecido, com redução de 45,8%. O redutor de plurais quase não erra, mas reduz apenas 15,6%.

Tabela 6 – Conflação de palavras por stemmer (palavras com frequência ≥ 5, stopwords NLTK)

Stemmer	Palavras	Radicais	Redução	Palavras/radical	Maior grupo
nenhum	31.913	31.913	0,0%	1,00	1
plural	31.913	26.928	15,6%	1,19	3
RSLP	31.913	15.286	52,1%	2,09	35
Snowball	31.913	17.281	45,8%	1,85	29

5.2.5 Granularidade e Hiperparâmetros da Representação

A unidade básica da representação é a receita completa, dividida em campos. Foram implementadas e comparadas três granularidades:

- **documento**: os cinco campos formam uma única unidade;
- **campos**: uma unidade por documento, mas cada termo recebe a indicação do campo em que aparece (por exemplo, `i:cebol` para "cebola" nos ingredientes e `p:cebol` no preparo), de modo que cada campo tem suas próprias listas;
- **trecho** (*chunk*): o documento é dividido em janelas de N termos, que podem se sobrepor.

A granularidade por campos foi escolhida pelo objetivo da busca e pelo tipo de documento. A busca por ingredientes precisa distinguir "frango" na lista de ingredientes de uma menção no texto ou em uma lista de receitas relacionadas, e os campos permitem restringir a busca ou dar pesos diferentes a cada um. Além disso, as receitas são textos curtos e sobre um único assunto (192 termos indexados em média), de modo que dividi-las em trechos aumenta o índice sem separar assuntos diferentes. Foram comparados trechos de 32, 64, 128, 256 e 512 termos sem sobreposição e trechos de 128 termos com sobreposição de 16, 32 e 64.

O tamanho dos lotes de processamento (*buckets*) é um parâmetro separado da segmentação do texto: é o número de ocorrências mantidas em memória durante a construção antes de serem gravadas em disco. Foram comparados lotes de 100 mil, 500 mil, 2 milhões e 5 milhões de postings (pares de termo e documento) e a construção sem limite, com todo o índice em memória.

Não foram aplicados limites de frequência: além das stopwords, nenhum termo raro ou frequente é removido, pois os termos raros incluem ingredientes pouco comuns. Também não há pesos por campo no índice; eles serão definidos no modelo de ordenação da próxima etapa.

5.2.6 Implementação do Índice Invertido

O índice invertido associa cada termo do vocabulário a uma lista de ocorrências (*postings*). Para cada documento em que o termo aparece, a lista guarda o identificador do documento, quantas vezes o termo aparece nele (*tf*) e em quais posições.

A construção usa o método SPIMI (*Single-Pass In-Memory Indexing*) (MANNING; RAGHAVAN; SCHÜTZE, 2008). Os documentos são lidos um a um e suas ocorrências são acumuladas em memória. Quando o lote atinge o limite, os termos são ordenados e o lote é gravado em disco. No fim, os lotes gravados são intercalados em um único índice. Assim, a memória usada não depende do tamanho da coleção.

O formato é ilustrado abaixo com um trecho real da lista de `i:cebol` (“cebola” nos ingredientes): no documento 2, “Como fazer pernil”, o termo aparece duas vezes, nas posições 14 e 30.

```
termo   -> [(documento, tf, [posicoes...]), ...]
i:cebol -> [(2, 2, [14, 30]), (5, 1, [30]), ...]
```

O índice é gravado em três arquivos: as listas de ocorrências, em formato binário; o vocabulário, com cada termo, o número de documentos em que aparece (*df*) e a posição de sua lista no arquivo binário; e uma tabela com o tamanho de cada documento, que será usada na ordenação. O vocabulário é carregado em memória, e cada lista é lida com um único acesso ao disco. Para ocupar menos espaço, cada número é guardado como a diferença em relação ao anterior, que é um número pequeno, e comprimido com o código Variable Byte (MANNING; RAGHAVAN; SCHÜTZE, 2008), que usa menos bytes para números menores. Um arquivo separado associa cada documento à URL, ao título, ao site e aos metadados, para exibir os resultados.

A implementação foi verificada por testes automatizados e por conferência manual. Os testes confirmam que o índice construído em vários lotes é idêntico ao construído de uma só vez e comparam 404 listas com uma contagem direta, feita sem o SPIMI; todos passam. Na conferência manual (Tabela 7), o número de documentos registrado no índice para termos frequentes, raros e ausentes foi comparado com uma contagem feita diretamente no corpus limpo, e os valores coincidiram em todos os casos.

Tabela 7 – Conferência da frequência de documentos no índice principal

Palavra	Termo	*df* no índice	Contagem direta	Primeiro documento
cebola (ingredientes)	`i:cebol`	22.662	22.662	Como fazer pernil
xícara (ingredientes)	`i:xic`	19.096	19.096	Pesto genovês
natas (ingredientes)	`i:nat`	4.868	4.868	Receita de torta de mousses
chávena (ingredientes)	`i:chaven`	1.665	1.665	Pão de uvas passas e cerveja preta
airfryer (preparo)	`p:airfry`	275	275	Como fazer cupim na AirFryer
feijoada (título)	`t:feijo`	240	240	Receita de vinagrete para feijoada
ornitorrinco (ingredientes)	`i:ornitorrinc`	0	0	–

O índice principal tem 81.432 termos e 14,3 milhões de ocorrências armazenadas, provenientes de 27,3 milhões de palavras; as demais são stopwords, números ou palavras de uma letra.

Por fim, foi implementada uma busca de demonstração, que encontra os documentos com todos os termos, reconhece expressões entre aspas pelas posições e ordena os resultados por TF-IDF. Consultas como `frango batata`, `"pão de queijo"` e `"leite condensado"coco` restrita aos ingredientes são respondidas em até 280 milissegundos, e os primeiros resultados são receitas pertinentes, como “Strogonoff de frango com batata”, “Pão de queijo mineiro” e “Cocada cremosa de leite condensado”. Essa ordenação serve apenas para validar o índice; o modelo de ordenação será definido na próxima etapa.

5.2.7 Procedimento Experimental e Medições

Os experimentos foram executados em um computador com processador AMD Ryzen 5 5600, 16 GB de memória e Windows 11, usando um único núcleo; as versões das bibliotecas estão no arquivo `requirements.txt` do código entregue. Todas as configurações usaram o mesmo corpus limpo, gerado a partir do `corpus.jsonl` identificado pelo SHA-256 registrado no manifesto.

Foram construídas 32 configurações, alterando um parâmetro por vez em relação a uma configuração de referência: granularidade por documento, posições armazenadas, stopwords do NLTK e do domínio, RSLP, sem acentos, sem números e lotes de 2 milhões de postings. Cada configuração rodou em um processo separado, para que a memória de uma não afetasse a outra. O tamanho do índice é sempre o mesmo para uma configuração, mas o tempo e a memória variam; por isso, cinco configurações representativas foram executadas três vezes cada.

O tempo foi medido em cinco fases: leitura do corpus, análise léxica, montagem do índice em memória, gravação dos lotes e intercalação final. A memória medida é a de todo o processo, e são registrados o pico e o acréscimo em relação ao início da indexação. O tamanho em disco soma os três arquivos do índice e não inclui os metadados usados para exibir os resultados.

A Tabela 8 apresenta as configurações repetidas, com a mediana das três execuções. Os resultados das 32 configurações estão no Apêndice A; como cada uma foi executada uma vez, diferenças de tempo abaixo de cerca de 10% entre elas não devem ser interpretadas.

Tabela 8 – Comparação das configurações de representação e indexação

Configuração	Tempo (s)	Pico (MiB)	Índice (MiB)	Termos
A: documento, sem stopwords, sem stemming, lotes de 2 M	84,9 (84,8–97,7)	88,6	57,8	78.221
B: documento, NLTK + domínio, RSLP, lotes de 2 M (referência)	81,0 (80,5–94,7)	83,2	38,3	40.985
C: campos, NLTK + domínio, RSLP, lotes de 2 M	90,1 (85,1–97,3)	93,0	39,9	81.432
D: trechos de 128 termos, NLTK + domínio, RSLP, lotes de 2 M	81,6 (81,3–85,2)	84,1	43,4	40.985
E: campos, NLTK + domínio, RSLP, lotes de 500 mil (escolhida)	86,5 (85,1–90,3)	83,0	39,9	81.432

Tempo: mediana de três execuções, com a menor e a maior entre parênteses. Pico: mediana da memória residente máxima do processo, que começa com cerca de 52 MiB. Todas as configurações são posicionais, sem acentos e sem números.

A memória variou menos de 0,5 MiB entre as repetições, e o tempo variou até 17%, porque a primeira rodada coincidiu com outro processamento na máquina; por isso, a mediana

é a medida usada. A configuração E usa 10 MiB a menos de memória que a C e gera o mesmo índice.

5.2.8 Análise dos Resultados e Limitações

Distribuição dos termos. Sem remoção de stopwords nem stemming, a coleção tem 27,3 milhões de tokens e 78.221 termos distintos, dos quais 28.200 aparecem uma única vez. A frequência dos termos segue aproximadamente a lei de Zipf (Figura 1): poucos termos concentram as ocorrências. O mais frequente, "de", ocorre 1,97 milhão de vezes e está em 74.172 dos 74.209 documentos, seguido por "com" e "para". Termos desse tipo têm listas de postings enormes e quase não distinguem os documentos, o que motiva a remoção de stopwords. O crescimento do vocabulário (Figura 2) segue a lei de Heaps, $V = Kn^{\beta}$, com $K = 30{,}8$ e $\beta = 0{,}465$, valores dentro da faixa usual para textos em linguagem natural (MANNING; RAGHAVAN; SCHÜTZE, 2008). Com esse expoente, dobrar a coleção aumentaria o vocabulário em cerca de 38%, e o léxico continuaria cabendo em memória.

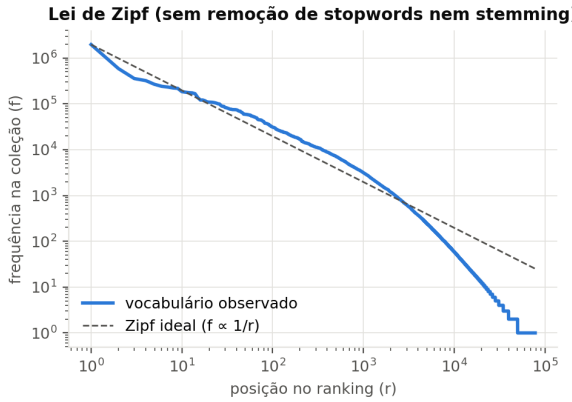


Figura 1 – Frequência dos termos em função da posição no ranking, em escala logarítmica

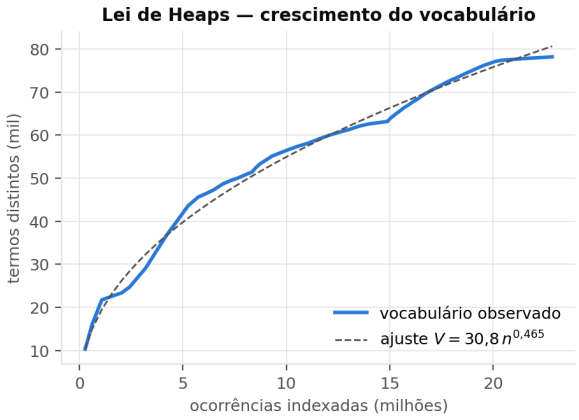


Figura 2 – Crescimento do vocabulário com o número de ocorrências indexadas

Stopwords e stemming. A configuração sem transformações apresentou o maior índice, com 57,8 MiB e 78.221 termos. A comparação com as demais (Figura 3; valores na Tabela 12, no Apêndice A) mostra que as duas transformações atuam sobre medidas diferentes. O stemming reduz o vocabulário: com o RSLP, ele cai para 40.985 termos (47,6% menos), contra 45.410 com o Snowball e 68.032 com o redutor de plurais. A remoção de stopwords quase não altera o vocabulário, pois a lista tem poucas palavras, mas reduz o índice, porque essas palavras têm as maiores listas: sem stemming, o índice cai para 45,6 MiB com a lista do NLTK e para 41,8 MiB com os termos do domínio. A combinação de stopwords do NLTK e do domínio com o RSLP gera o menor índice, com 38,3 MiB, 33,7% menor que sem transformações. Manter os acentos aumenta o vocabulário em 6,5%, e indexar os números o aumenta em 21,8%. O tempo de indexação quase não depende dessas escolhas.

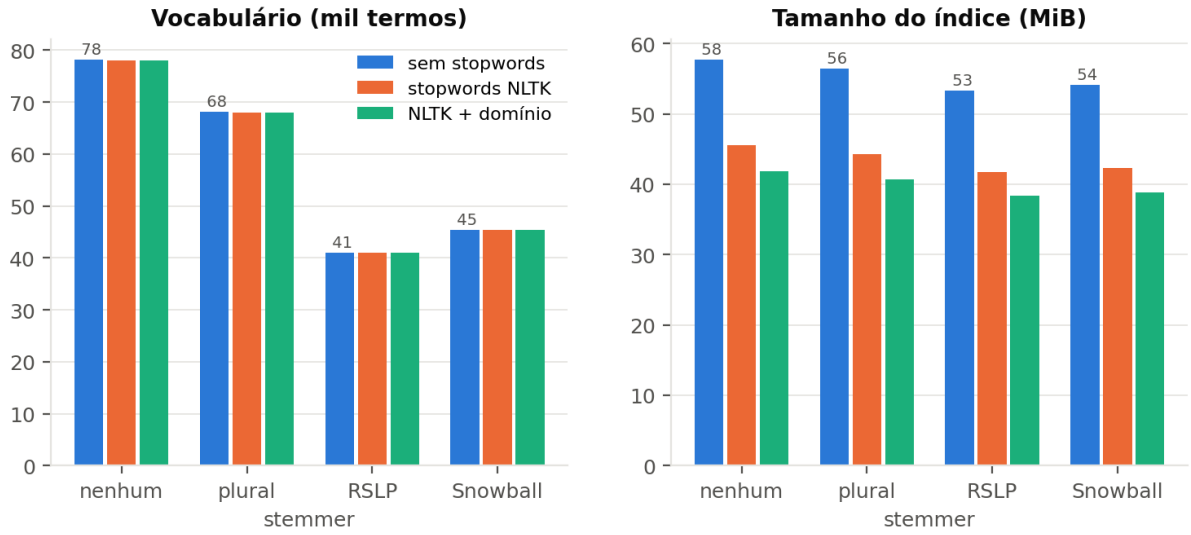


Figura 3 – Vocabulário e tamanho do índice para cada combinação de stopwords e stemmer

Granularidade e tamanho do trecho. Quanto menor o trecho, mais unidades e maior o índice (Figura 4; valores na Tabela 13, no Apêndice A). Com 32 termos, os 74.209 documentos viram 482.320 unidades, e o índice cresce de 38,3 para 57,0 MiB, porque um termo que aparece em vários trechos do mesmo documento passa a ter um posting em cada um. Com 512 termos, quase todo documento cabe em um único trecho (77.273 unidades e 38,9 MiB). A sobreposição repete termos e aumenta ainda mais o índice: com trechos de 128 termos, ele passa de 43,4 MiB sem sobreposição para 66,0 MiB com sobreposição de 64. O índice por campos dobra o vocabulário (81.432 termos), já que o mesmo radical aparece com prefixos diferentes, e aumenta o número de postings de 7,45 para 9,52 milhões, mas ocupa apenas 4% a mais que o índice por documento (39,9 MiB contra 38,3 MiB).

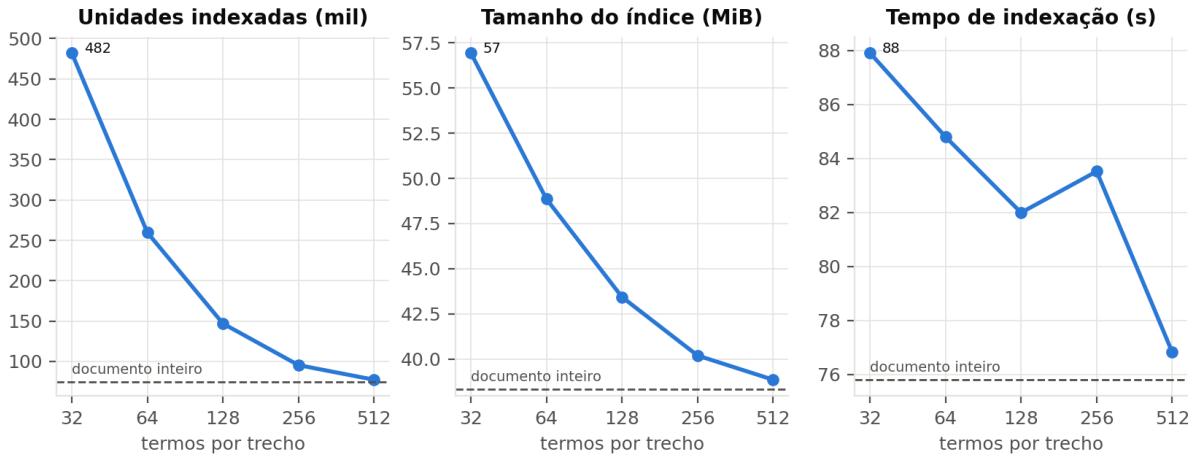


Figura 4 – Efeito do tamanho do trecho sobre o número de unidades, o tamanho do índice e o tempo; a linha tracejada é o índice por documento

Tamanho dos lotes. O tamanho do lote (Figura 5; valores na Tabela 14, no Apêndice A) determina a memória usada na construção. Sem limite, todo o índice fica em memória e o

processo usa 91 MiB além do inicial. Com lotes de 500 mil postings, são gravados 20 lotes e o acréscimo cai para 31 MiB. Lotes menores geram mais arquivos para intercalar: com 100 mil postings são 96 lotes, e a intercalação leva 2,9 segundos, contra 0,3 segundo com um único lote. O tempo total ficou entre 92 e 100 segundos, dentro da variação entre execuções. Esse grupo foi executado novamente depois da correção de um excesso de memória na intercalação; por isso, seus tempos devem ser comparados apenas entre si.

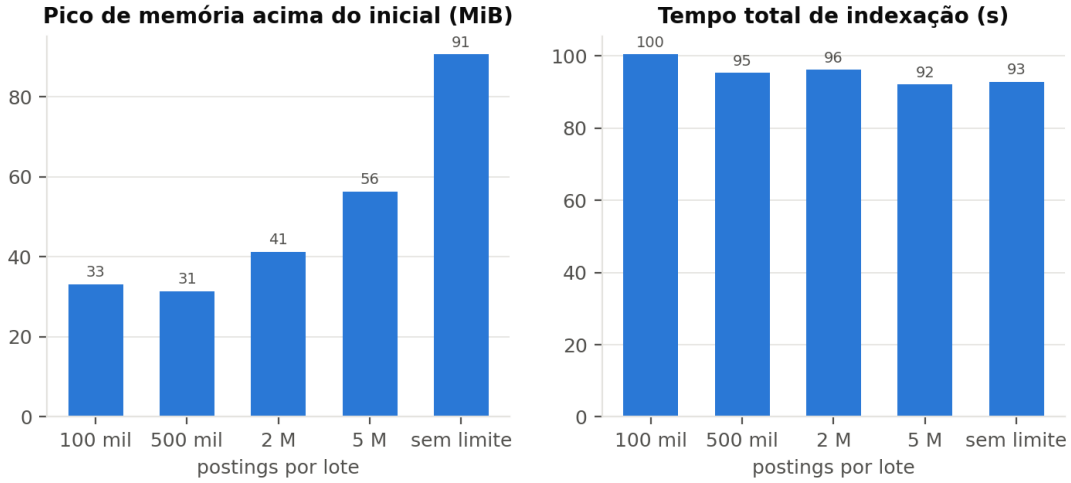


Figura 5 – Pico de memória e tempo total em função do limite de postings por lote

Posições, compressão e escopo. As posições ocupam a maior parte das listas (Tabela 9): sem elas, o índice por documento cai de 38,3 para 16,8 MiB, e o índice por campos de 39,9 para 23,1 MiB. Como as posições são necessárias para buscar expressões comuns em receitas, como "leite condensado" e "pão de queijo", elas foram mantidas. O Variable Byte reduz as listas a um tamanho entre 1/3,9 e 1/3,1 do que ocupariam com inteiros de 32 bits. Indexar somente as 66.872 receitas reduz o índice por campos para 32,4 MiB.

Tabela 9 – Espaço ocupado pelo índice: posições, compressão VByte e escopo

Configuração	Documentos	Vocabulário	Postings VByte (MiB)	Postings int32 (MiB)	Compressão	Léxico (MiB)	Unidades (MiB)	Total (MiB)
Documento, posicional	74.209	40.985	36,1	111,3	3,08×	0,9	1,2	38,3
Documento, sem posições	74.209	40.985	14,7	56,8	3,87×	0,9	1,2	16,8
Campos, posicional	74.209	81.432	36,0	127,2	3,54×	2,0	1,9	39,9
Campos, sem posições	74.209	81.432	19,2	72,7	3,78×	1,9	1,9	23,1
Campos, só receitas	66.872	69.893	29,0	105,5	3,64×	1,7	1,7	32,4

Tempo por fase. A análise léxica consome a maior parte do tempo de indexação (Figura 6): 59,6 dos 75,8 segundos no índice por documento (79%) e 65,9 dos 88,4 segundos no índice por campos (75%). A montagem do índice em memória leva de 13 a 21 segundos, e a gravação dos lotes com a intercalação leva menos de um segundo. O gargalo, portanto, é o processamento do texto em Python, e não o acesso ao disco. Se for necessário reindexar mais rápido, o caminho é distribuir a análise léxica entre vários processos.

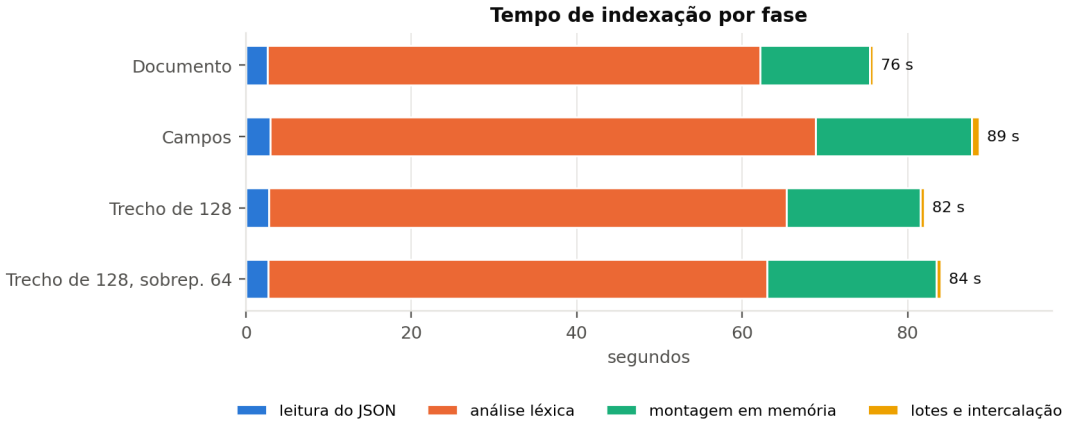


Figura 6 – Tempo de indexação por fase para quatro granularidades

A escolha para a próxima fase é o índice por campos, posicional, com stopwords do NLTK e do domínio, RSLP, sem acentos e sem números. O motivo é que essa combinação mantém o índice perto do menor tamanho medido (39,9 MiB contra 38,3 MiB do índice por documento) e preserva duas capacidades centrais para a busca por ingredientes: restringir ou ponderar a busca por campo e reconhecer expressões. A construção passa a usar lotes de 500 mil postings, que tiveram o menor pico de memória; o índice resultante é idêntico ao construído com qualquer outro tamanho de lote, como verificado nos testes. A escolha do stemmer é provisória: o RSLP produz o menor índice, mas reúne palavras de significados diferentes, e será comparado ao redutor de plurais pela qualidade da busca.

As limitações identificadas são:

- **perda de informação na limpeza**: a remoção de boilerplate pode retirar frases legítimas que se repetem em muitas páginas de um site, e o corte do corpo nos marcadores de links descarta o texto posterior;
- **classificação heurística**: a marcação de receitas não foi avaliada manualmente de forma sistemática, e alguns posts de blog com uma receita embutida são marcados como receita, como “Pão de queijo engorda ou é saudável?”, que aparece entre os primeiros resultados da consulta `"pão de queijo"`;
- **aproximações do stemming**: o RSLP reúne “leite” e “leitura”, “cebola” e “cebolinha” e “caseiro” e “casa”;
- **negações removidas**: “sem” e “não” estão na lista de stopwords, o que impede consultas como “bolo sem ovo”;
- **campos ausentes**: na primeira coleta, ingredientes e preparo dependem dos marcadores de cada site, e os metadados não estão disponíveis para todas as páginas (por exemplo, o tempo de preparo foi obtido para 54.554 dos 74.209 documentos);
- **conteúdo repetido**: a deduplicação identifica somente cópias exatas, de modo que a mesma receita publicada com pequenas alterações em dois sites permanece duplicada;

- **variantes do português**: a coleção mistura português do Brasil e de Portugal, e sinônimos como “creme de leite” e “natas” ou “xícara” e “chávena” ainda não são unificados. A grafia também varia, como “airfryer” e “air fryer”.

Melhor desempenho de indexação não é apresentado como prova de melhor qualidade de busca: um índice menor ou mais rápido de construir pode recuperar resultados piores. A qualidade das configurações será avaliada na próxima etapa, com consultas e julgamentos de relevância.

5.3 Próximos Passos e Cronograma de Conclusão

Com o índice construído, a próxima etapa é o processamento de consultas e a avaliação da busca. Os próximos passos são:

1. **Modelo de ordenação**: implementar o BM25 e sua variante por campos, BM25F, usando os comprimentos por campo já registrados em `unidades.tsv`, e ajustar os pesos de título, ingredientes, preparo e corpo.

2. **Processamento de consultas**: aplicar à consulta o mesmo analisador dos documentos, com suporte a expressões, termos obrigatórios e busca restrita aos ingredientes. Isso inclui retirar "sem" e "não" da lista de stopwords e tratar a negação, como em "bolo sem ovo".

3. **Filtros**: usar os metadados extraídos (tempo de preparo, dificuldade, rendimento e categoria) como filtros e completar a extração nos sites com menor cobertura.

4. **Vocabulário**: unificar sinônimos do português do Brasil e de Portugal e variantes de grafia, e decidir o stemmer de cada campo com base na avaliação.

5. **Avaliação**: montar um conjunto de consultas típicas (por exemplo, "frango com batata" e "bolo sem ovo"), julgar a relevância dos primeiros resultados e calcular precisão nos dez primeiros resultados (P@10), MAP e nDCG para comparar as configurações.

6. **Interface**: construir uma interface de busca que exiba título, ingredientes, tempo de preparo e link para a receita original.

As tarefas estão organizadas conforme a Tabela 10, que registra as atividades concluídas nesta etapa e as pendentes, com os responsáveis. Como o prazo da entrega final ainda não foi definido, as atividades pendentes estão indicadas em semanas contadas a partir desta entrega (S1 inicia em 23/10/2026).

Tabela 10 – Cronograma e divisão das tarefas para conclusão do projeto

Atividade e entrega	Responsável(is)	Início e término	Situação
Limpeza e validação do corpus	Todos os integrantes	até 18/10/2026	Concluída
Tokenização, stopwords e stemming	Todos os integrantes	até 18/10/2026	Concluída; ajuste das negações em S1
Índice invertido e persistência em arquivos	Todos os integrantes	até 18/10/2026	Concluída
Experimentos e análise de hiperparâmetros	Todos os integrantes	até 18/10/2026	Concluída
Relatório parcial e entrega do código-fonte	Todos os integrantes	18/10 a 22/10/2026	Em andamento
Ordenação BM25/BM25F e processamento de consultas	Gustavo e Josué	S1 a S3	Pendente
Filtros por metadados e sinônimos	Luan e Josué	S2 a S4	Pendente
Consultas de teste e julgamentos de relevância	Ana Beatriz e Nicole	S1 a S3	Pendente
Avaliação da busca (P@10, MAP, nDCG) e ajustes	Ana Beatriz e Gustavo	S4 e S5	Pendente
Interface de busca	Nicole e Luan	S3 a S5	Pendente
Relatório final, revisão e entrega do código-fonte	Todos os integrantes	S5 e S6	Pendente

A previsão é concluir o desenvolvimento em seis semanas a partir desta entrega; a data final será ajustada quando o prazo da entrega final for definido. As dependências entre as tarefas são: o modelo de ordenação precisa estar disponível antes da avaliação; as consultas de teste e os julgamentos de relevância podem ser preparados em paralelo à ordenação, pois dependem apenas do corpus limpo; e a interface depende do processamento de consultas e dos filtros. A participação de Ana Beatriz, Gustavo, Josué, Luan e Nicole está distribuída entre as atividades pendentes, com pelo menos duas atividades por integrante.

Para a entrega desta etapa, foram reunidos na pasta `parte2/`: o código-fonte das quatro etapas e dos testes; as instruções de execução, em `LEIA_ME.md`; as versões das dependências, em `requirements.txt`; as configurações e métricas de cada experimento, em `resultados/experimentos.csv` e `resultados/repeticoes.csv`; as figuras e tabelas deste relatório; e a identificação do corpus utilizado, pelo SHA-256 do manifesto. Os arquivos gerados (corpus limpo e índices) podem ser reproduzidos com os comandos do `LEIA_ME.md`. Este relatório distingue as funcionalidades implementadas, descritas na Seção 5.2, das atividades previstas para a conclusão do sistema, listadas acima.

REFERÊNCIAS

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge: Cambridge University Press, 2008.

ORENGO, V. M.; HUYCK, C. A stemming algorithm for the Portuguese language. In: **Proceedings of the Eighth International Symposium on String Processing and Information Retrieval (SPIRE 2001)**. Laguna de San Rafael: IEEE, 2001. p. 186–193.

PORTER, M. F. **Snowball: A language for stemming algorithms**. 2001. Disponível em: https://snowballstem.org/texts/introduction.html.

APÊNDICE A – RESULTADOS DETALHADOS

Este apêndice apresenta os valores completos da limpeza por fonte e das configurações comparadas nos experimentos, resumidos no texto pelas Figuras 3, 4 e 5. Os mesmos dados estão em `resultados/experimentos.csv`, no código entregue.

Tabela 11 – Resultado da limpeza e deduplicação por fonte

Fonte	Entrada	Duplicatas	Mantidos	Receitas	Texto bruto (M caract.)	Texto limpo (M caract.)	Com tempo de preparo
Petitchef	29.074	0	29.074	29.074	12,3	32,2	28.697
Receitas Globo	15.906	1.417	14.489	9.009	121,7	41,4	4.031
TudoReceitas	11.062	1.023	10.039	8.993	72,4	41,5	8.873
TeleCulinária	10.913	5	10.908	10.908	35,9	18,4	10.374
Panelinha	10.210	7.273	2.937	2.875	34,7	9,1	0
Estadão Paladar	2.675	0	2.675	1.932	9,9	6,2	0
TudoGostoso	2.561	0	2.561	2.556	21,6	3,4	2.541
Receitas de Minuto	1.502	6	1.496	1.496	4,1	3,7	29
Terra Degusta	21	0	21	20	0,2	0,1	0
Ana Maria Brogui	9	0	9	9	0,0	0,0	9
Total	83.933	9.724	74.209	66.872	312,8	156,0	54.554

Tabela 12 – Efeito das transformações léxicas (granularidade documento, índice posicional)

Stopwords	Stemmer	Vocabulário	Ocorrências (milhões)	Postings (milhões)	Índice (MiB)	Tempo total (s)	Análise léxica (s)
sem stopwords	nenhum	78.221	22,92	10,72	57,8	95,0	67,1
sem stopwords	plural	68.119	22,92	10,38	56,4	80,5	58,4
sem stopwords	RSLP	41.001	22,92	9,63	53,4	85,8	63,5
sem stopwords	Snowball	45.442	22,92	9,84	54,1	87,6	64,3
stopwords NLTK	nenhum	78.098	15,85	8,99	45,6	87,0	64,4
stopwords NLTK	plural	68.032	15,85	8,66	44,3	86,0	64,2
stopwords NLTK	RSLP	40.986	15,85	8,08	41,8	78,4	60,6
stopwords NLTK	Snowball	45.410	15,85	8,21	42,3	82,4	63,2
NLTK + domínio	nenhum	78.075	14,28	8,23	41,8	79,4	60,2
NLTK + domínio	plural	68.023	14,28	7,94	40,7	80,5	61,4
NLTK + domínio	RSLP	40.985	14,28	7,45	38,3	82,2	64,0
NLTK + domínio	Snowball	45.409	14,28	7,58	38,9	82,6	64,0
NLTK + domínio	RSLP, mantendo acentos	43.667	14,28	7,48	38,5	78,4	61,2
NLTK + domínio	RSLP, indexando números	49.906	15,76	8,23	42,3	81,3	62,2

Tabela 13 – Granularidade da representação (NLTK + domínio, RSLP, posicional)

Granularidade	Sobre-posição	Unidades	Vocabulário	Postings (milhões)	Posições (milhões)	Índice (MiB)	Tempo (s)	RAM extra (MiB)
Documento	–	74.209	40.985	7,45	14,28	38,3	75,8	32
Campos (zonas)	–	74.209	81.432	9,52	14,28	39,9	88,4	41
Trecho de 32	0	482.320	40.985	11,52	14,28	57,0	87,9	31
Trecho de 64	0	259.763	40.985	10,26	14,28	48,9	84,8	31
Trecho de 128	0	146.938	40.985	9,00	14,28	43,4	82,0	32
Trecho de 256	0	95.379	40.985	8,10	14,28	40,2	83,5	32
Trecho de 512	0	77.273	40.985	7,67	14,28	38,9	76,8	31
Trecho de 128	16	153.232	40.985	9,84	15,55	47,2	82,9	31
Trecho de 128	32	161.836	40.985	10,85	17,09	51,8	84,7	32
Trecho de 128	64	192.447	40.985	13,95	21,85	66,0	83,9	32

Tabela 14 – Tamanho do lote na construção SPIMI (índice por campos, posicional)

Postings por lote	Lotes	RAM extra (MiB)	Montagem em memória (s)	Escrita dos lotes (s)	Intercalação (s)	Total (s)
100 mil	96	33,2	22,1	1,31	2,88	100,4
500 mil	20	31,4	21,0	0,62	1,03	95,3
2 M	5	41,3	21,4	0,39	0,53	96,1
5 M	2	56,4	20,3	0,29	0,37	92,1
sem limite	1	90,7	20,4	0,00	0,28	92,8